

FCM I Programming Assignment 4

Charles Ohanian

December 9, 2024

1 Executive Summary

In this report, we explore the implementation, behavior, and performance of several root-finding methods: Regula Falsi, Secant, Newton's method (both standard and modified), and Steffensen's method. Empirical experiments examined their convergence behavior for polynomials with roots of varying multiplicities and specific configurations. Theoretical predictions about convergence rates were tested through numerical experiments, revealing how initial guesses, root multiplicities, and scaling factors influence method performance.

2 Statement of the Problem

The assignment focuses on implementing and analyzing several numerical methods for solving nonlinear equations, including Regula Falsi, Secant, Newton's (standard and modified), and Steffensen's methods. These root-finding methods are fundamental tools in scientific computing, engineering, and applied mathematics, where closed-form solutions are often unavailable. They are widely used in fields such as optimization, control systems, and numerical simulation.

The primary goal is to evaluate the performance of these methods through targeted experiments involving polynomial functions with roots of varying multiplicities and specific configurations. The assignment investigates how initial guesses, root multiplicities, and scaling factors affect convergence rates. Particular attention is given to Newton's method's cycling behavior near critical points, numerical stability, and comparative performance with other methods.

Additionally, we explore how root-finding methods behave when distinct roots approach coalescence, highlighting their strengths and limitations. Understanding these methods' theoretical and practical aspects helps develop robust algorithms for real-world applications where numerical precision and computational efficiency are critical.

3 Description of the Mathematics

Root-finding methods rely on iterative algorithms that approximate solutions to equations of the form $f(x) = 0$. These methods are based on constructing local linear approximations

of a nonlinear function. Since finding the root of a linear function is straightforward, these methods iteratively approximate $f(x)$ by a linear model $l_k(x)$ near the current estimate x_k :

$$l_k(x) = f(x_k) + (x - x_k)q_k.$$

The slope q_k is chosen differently for each method, determining how the next approximation x_{k+1} is computed. The general update rule is:

$$x_{k+1} = x_k - \frac{f(x_k)}{q_k}.$$

Regula Falsi maintains two endpoints a_k and b_k , ensuring that $f(a_k)f(b_k) < 0$, meaning the root is bracketed. The slope, to be plugged into the general formula above, is determined by the secant line through these points:

$$q_k = \frac{f(b_k) - f(a_k)}{b_k - a_k}.$$

Depending on the sign of $f(x_{k+1})$, either a_k or b_k is updated to maintain the bracketing property. The method guarantees convergence but can be slow if one endpoint becomes fixed for many iterations.

The Secant method uses two consecutive points, x_{k-1} and x_k , to approximate the derivative:

$$q_k = \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}.$$

The next iterate is computed using the general update rule above. The method is not guaranteed to converge, as it does not enforce bracketing. However, it typically converges faster than Regula Falsi, with a superlinear convergence rate of approximately 1.618.

Newton's method uses the derivative of the function as the slope: $q_k = f'(x)$. x_{k+1} is again computed using the general formula above. This method achieves quadratic convergence near simple roots, assuming a sufficiently close initial guess. For roots with multiplicity d , the standard method has linear convergence, but a modified version with the update:

$$x_{k+1} = x_k - \frac{f(x_k)}{mf'(x_k)}$$

restores quadratic convergence when $m = d$. When derivatives are small, numerical stability can be an issue, potentially causing divergence.

Steffensen's method approximates the slope using:

$$q_k = \frac{f(x_k + f(x_k)) - f(x_k)}{f(x_k)}.$$

The next iterate is computed with the same general update formula. This method achieves quadratic convergence, like Newton's method, without explicitly computing the derivative. This makes it useful when derivative computations are impractical.

4 Description of the Algorithm and Implementation

Five distinct algorithms were developed for Regula Falsi, Secant, Standard Newton's, Modified Newton's, and Steffensen's method. All routines were developed and run in Spyder, an open source python environment included within Anaconda.

4.1 Regula Falsi Method

The Regula Falsi function takes several input parameters, including the function f , the initial interval endpoints a and b , the true root ρ for analysis purposes, a convergence tolerance ϵ , and a maximum iteration limit. If the function values at the endpoints $f(a)$ and $f(b)$ do not have opposite signs, indicating the possible absence of a root in the initial interval, the function raises an error. Upon initialization, the slope q of the secant line connecting $(a, f(a))$ and $(b, f(b))$ is computed, and the initial guess for the root is set to $x = b$ (chosen arbitrarily instead of $x = a$). The iteration counter k is initialized to zero, and a list is created to store relevant data from each iteration, including the iteration count, the computed root approximation, the change in the root estimate, and the relative error. The algorithm proceeds through a loop that continues until the method converges or the maximum number of iterations is reached. In each iteration, the next root approximation is computed using the update formula $x_{\text{next}} = x - f(x)/q$. The absolute error $|x_{\text{next}} - \rho|$ and the change $\Delta x = x_{\text{next}} - x$ are calculated, along with the relative error compared to the previous estimate. If the absolute error falls below the specified tolerance ϵ , the function returns the computed root and the iteration log. To ensure the root remains within the updated interval, the endpoints are adjusted based on the sign of $f(x_{\text{next}})$; either a or b is updated accordingly. The slope q is recalculated based on the new interval, and the current estimate x is updated to the newly computed x_{next} . The iteration counter is then incremented. If the function reaches the maximum number of iterations without satisfying the convergence criterion, an exception is raised, indicating failure to converge.

4.2 Secant Method

The Secant Method function takes as input the function f , two initial guesses x_0 and x_1 , the true root ρ for analysis purposes, a convergence tolerance ϵ , and a maximum iteration limit. An iteration counter k is initialized to zero, and a list is created to store relevant data from each iteration, including the iteration count, the computed root approximation, the change in the root estimate, and the relative error. The algorithm proceeds by calculating the slope q_k of the secant line connecting the points $(x_0, f(x_0))$ and $(x_1, f(x_1))$. Using this slope, the next approximation for the root is computed using the update formula $x_{\text{next}} = x_1 - f(x_1)/q$. The change in the root approximation $\Delta x = x_{\text{next}} - x_1$ is computed, along with the absolute error $|x_{\text{next}} - \rho|$ and the relative error, defined as the ratio of the new error to the previous error. This data is stored in the iteration log for future analysis. If the absolute error falls below the specified tolerance ϵ , the function returns the computed root approximation and the iteration log. If convergence is not achieved, the algorithm updates the previous guesses by setting x_0 to x_1 and x_1 to x_{next} . The iteration counter k is then incremented. If the function reaches the maximum number of iterations without satisfying the convergence criterion, an exception is raised, indicating failure to converge.

4.3 Newton's Method (Standard and Modified)

The Standard Newton's method function takes several input parameters, including the function f , the derivative df , the initial guess x_0 , the true root ρ for analysis purposes, a convergence tolerance ϵ , and a maximum iteration limit. The iteration counter k is initialized to zero, and the initial guess for the root is set to $x = x_0$. A list is created to store data from each iteration, including the iteration count, the computed root approximation, the change in the root estimate, and the relative error. The algorithm proceeds by evaluating the derivative at the current point, $q_k = f'(x)$. If the derivative is zero, an error is raised because division by zero would make the method undefined. Assuming $q_k \neq 0$, the next approximation for the root is computed using the update formula $x_{\text{next}} = x - f(x)/q$. The change in the root estimate $\Delta x = x_{\text{next}} - x$, the absolute error $|x_{\text{next}} - \rho|$, and the relative error are calculated. This data is logged for future analysis. If the absolute error falls below the specified tolerance ϵ , the function returns the computed root approximation and the iteration log. If convergence is not achieved, the current estimate x is updated to x_{next} , and the iteration counter k is incremented. If the function reaches the maximum number of iterations without satisfying the convergence criterion, an exception is raised, indicating failure to converge.

The Modified Newton's method function is very similar. It takes an additional input parameter, m , which is the multiplicity of the root. Instead of using the iteration step in Standard Newton's, the function uses the update formula $x_{\text{next}} = x - m(f(x)/q)$.

4.4 Steffensen's Method

The Steffensen's Method function accepts as inputs the function f , an initial guess x_0 , the true root ρ for analysis purposes, a convergence tolerance ϵ , and a maximum iteration limit. The iteration counter k is initialized to zero, and the initial guess is set to $x = x_0$. A list is created to store details from each iteration, including the iteration count, the computed root approximation, the change in the root estimate, and the relative error. During each iteration, the function value at the current point x , denoted by $f(x)$, is computed. A shifted function value $f(x + f(x))$ is then evaluated and used to compute the derivative-like quantity $\text{theta} = \frac{f(x + f(x)) - f(x)}{f(x)}$. The next root approximation is computed using the update formula $x_{\text{next}} = x - f(x)/\text{theta}$. The change in the root estimate $\Delta x = x_{\text{next}} - x$, the absolute error $|x_{\text{next}} - \rho|$, and the relative error are calculated and logged. If the absolute error falls below the specified tolerance ϵ , the function returns the computed root approximation and the iteration log. If convergence is not achieved, the current estimate x is updated to x_{next} , and the iteration counter k is incremented. If the function reaches the maximum number of iterations without satisfying the convergence criterion, an exception is raised, indicating failure to converge.

5 Description of the Experimental Design and Results

5.1 Higher Order Roots

Consider problems of the form $f(x) = (x - \rho)^d$, i.e., a single root at ρ of multiplicity d . For all methods, the absolute error of the computed root is chosen as the convergence criterion. That is, the method is said to converge in k iterations if $|x_k - \rho| < \epsilon$. This choice of convergence avoids the issues that arise with other choices of convergence. For instance, when using $f(x_k)$ as the convergence criterion, the methods may "converge" to roots but the computed root may actually be some ways away from the real root. This happens especially when working with higher multiplicities, as the function becomes flat in the neighborhood of the x-axis. Using this convergence criterion can lead to inaccurate comparisons. Thus, in all methods, the absolute error $|x_k - \rho|$ is used for convergence checking.

5.1.1 Question 1

Using standard Newton's method, i.e., $m = 1$, we see linear convergence for all $d > 1$. Figures 1 and 2 show the results of Standard Newton's method on the function $f(x) = (x - 4)^2$ with $x_0 = 2$ and $x_0 = 9$. The linear convergence is made clear by the constant relative error of 0.5. This constant is not random - it is equal to $1 - \frac{1}{d}$ (in this case, $1 - \frac{1}{2} = 0.5$). The computed x is also converging to 4 relatively slowly.

Iteration	x	Delta x	Relative Error
1	3.000000000000	1.000000000000	0.500000000000
2	3.500000000000	0.500000000000	0.500000000000
3	3.750000000000	0.250000000000	0.500000000000
4	3.875000000000	0.125000000000	0.500000000000
5	3.937500000000	0.062500000000	0.500000000000
...			
24	3.999999880791	0.000000119209	0.500000000000
25	3.999999940395	0.000000059605	0.500000000000
26	3.999999970198	0.000000029802	0.500000000000
27	3.999999985099	0.000000014901	0.500000000000
28	3.999999992549	0.000000007451	0.500000000000

Iteration	x	Delta x	Relative Error
1	6.500000000000	-2.500000000000	0.500000000000
2	5.250000000000	-1.250000000000	0.500000000000
3	4.625000000000	-0.625000000000	0.500000000000
4	4.312500000000	-0.312500000000	0.500000000000
5	4.156250000000	-0.156250000000	0.500000000000
...			
25	4.000000149012	-0.000000149012	0.500000000000
26	4.000000074506	-0.000000074506	0.500000000000
27	4.000000037253	-0.000000037253	0.500000000000
28	4.000000018626	-0.000000018626	0.500000000000
29	4.000000009313	-0.000000009313	0.500000000000

Figures 1 & 2. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Standard Newton's method with the following parameters: $\rho = 4$, $d = 2$, $x_0 = 2$ (left), $x_0 = 9$ (right), $\epsilon = 1 \times 10^{-8}$.

Figures 3 and 4 show the results of Standard Newton's method on the function $f(x) = (x - 4)^5$ with $x_0 = 3$ and $x_0 = 7$. We again see a near constant relative error of 0.8, with the only small deviations likely coming from machine error. This relative error follows the same relationship as above: $1 - \frac{1}{5} = 0.8$. It can be seen that although the method still converges here, it converges much slower than the case where $d = 2$, taking almost three times the number of iterations.

Iteration	x	Delta x	Relative Error
1	3.200000000000	0.200000000000	0.800000000000
2	3.360000000000	0.160000000000	0.800000000000
3	3.488000000000	0.128000000000	0.800000000000
4	3.590400000000	0.102400000000	0.800000000000
5	3.672320000000	0.081920000000	0.800000000000
...			
79	3.999999977914	0.00000005521	0.800000000000
80	3.999999982332	0.00000004417	0.799999995978
81	3.999999985865	0.00000003534	0.799999994973
82	3.999999988602	0.00000002827	0.800000012567
83	3.999999990954	0.00000002262	0.800000000000

Iteration	x	Delta x	Relative Error
1	6.400000000000	-0.600000000000	0.800000000000
2	5.920000000000	-0.480000000000	0.800000000000
3	5.536000000000	-0.384000000000	0.800000000000
4	5.228800000000	-0.307200000000	0.800000000000
5	4.983040000000	-0.245760000000	0.800000000000
...			
84	4.000000021711	-0.00000005428	0.799999993455
85	4.000000017369	-0.00000004342	0.800000008182
86	4.000000013895	-0.00000003474	0.800000000000
87	4.000000011116	-0.00000002779	0.800000012784
88	4.000000008893	-0.00000002223	0.799999984020

Figures 3 & 4. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Standard Newton's method with the following parameters: $\rho = 4$, $d = 5$, $x_0 = 3$ (left), $x_0 = 7$ (right), $\epsilon = 1 \times 10^{-8}$.

Figures 5 and 6 show the results of Standard Newton's method on the function $f(x) = (x - 4)^{10}$ with $x_0 = 1$ and $x_0 = 8$. Linear convergence can again be observed through near constant relative error of 0.9, with small deviations only coming from machine error. We again see the same relationship: $1 - \frac{1}{10} = 0.9$. In addition, pushing d up to 10 makes the convergence even slower.

Iteration	x	Delta x	Relative Error	Iteration	x	Delta x	Relative Error
1	1.300000000000	0.300000000000	0.900000000000	1	7.600000000000	-0.400000000000	0.900000000000
2	1.570000000000	0.270000000000	0.900000000000	2	7.240000000000	-0.360000000000	0.900000000000
3	1.813000000000	0.243000000000	0.900000000000	3	6.916000000000	-0.324000000000	0.900000000000
4	2.031700000000	0.218700000000	0.900000000000	4	6.624400000000	-0.291600000000	0.900000000000
5	2.228530000000	0.196830000000	0.900000000000	5	6.361960000000	-0.262440000000	0.900000000000
...				...			
182	3.999999985899	0.00000001567	0.900000011337	184	4.000000015229	-0.00000001692	0.900000005249
183	3.999999987309	0.00000001410	0.899999996851	185	4.000000013706	-0.00000001523	0.899999998336
184	3.999999988578	0.00000001269	0.8999999989502	186	4.000000012336	-0.00000001371	0.9000000025920
185	3.999999989720	0.00000001142	0.8999999988336	187	4.000000011102	-0.00000001234	0.8999999985600
186	3.999999990748	0.00000001028	0.8999999982720	188	4.000000009992	-0.00000001110	0.9000000016000

Figures 5 & 6. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Standard Newton's method with the following parameters: $\rho = 4$, $d = 5$, $x_0 = 1$ (left), $x_0 = 8$ (right), $\epsilon = 1 \times 10^{-8}$.

These tests show that Standard Newton's converges linearly for roots with multiplicity greater than 1. The higher the multiplicity, the slower the convergence, as the relative error approaches 1.

5.1.2 Question 2

Using Modified Newton's method, i.e., $m = d$, we see convergence in one step for all $d > 1$. Figures 7-12 show the results of Modified Newton's method on the function $f(x) = (x - 3.5)^d$ with $d = 2$, $d = 7$, and $d = 15$ and varying initial guesses. In all cases, the method jumps to the exact root in one iteration.

Iteration	x	Delta x	Relative Error	Iteration	x	Delta x	Relative Error
1	3.500000000000	0.500000000000	-0.000000000000	1	3.500000000000	-8.500000000000	0.000000000000

Figures 7 & 8. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Modified Newton's method with the following parameters: $\rho = 3.5$, $d = 2$, $x_0 = 3$ (left), $x_0 = 12$ (right), $\epsilon = 1 \times 10^{-8}$.

Iteration	x	Delta x	Relative Error	Iteration	x	Delta x	Relative Error
1	3.500000000000	1.500000000000	-0.000000000000	1	3.500000000000	-4.500000000000	0.000000000000

Figures 9 & 10. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Modified Newton's method with the following parameters: $\rho = 3.5$, $d = 7$, $x_0 = 2$ (left), $x_0 = 8$ (right), $\epsilon = 1 \times 10^{-8}$.

Iteration	x	Delta x	Relative Error	Iteration	x	Delta x	Relative Error
1	3.500000000000	0.500000000000	-0.000000000000	1	3.500000000000	-6.500000000000	0.000000000000

Figures 11 & 12. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Modified Newton's method with the following parameters: $\rho = 3.5$, $d = 15$, $x_0 = 3$ (left), $x_0 = 10$ (right), $\epsilon = 1 \times 10^{-8}$.

5.1.3 Question 3

Using Steffensen's method, the observed convergence trends are not as consistent. Figures 13 and 14 show the results of Steffensen's method on the function $(x - 6)^2$ with $x_0 = 5.2$ and $x_0 = 11$. Convergence is achieved, but the rate appears to be linear, rather than quadratic. There is no sudden drop off in distance to the true root and the relative error approaches a constant, 0.5.

Iteration	x	Delta x	Relative Error
1	5.86666666667	0.66666666667	0.16666666667
2	5.938095238095	0.071428571429	0.464285714286
3	5.970036270036	0.031941031941	0.484029484029
4	5.985246005217	0.015209735181	0.492395132410
5	5.992677827139	0.007431821922	0.496284089040
...			
14	5.999985803139	0.000014197260	0.499992957711
15	5.999992901623	0.000007098483	0.499996270108
16	5.999996450804	0.000003549181	0.500001097839
17	5.999998225371	0.000001774568	0.500008602892
18	5.999999112638	0.000000887266	0.500027092606

Iteration	x	Delta x	Relative Error
1	10.285714285714	-0.714285714286	0.857142857143
2	9.603896103896	-0.681818181818	0.840909090909
3	8.960790657778	-0.643105446118	0.821552723059
4	8.363952197535	-0.596838460243	0.798419230122
5	7.822252359743	-0.541699837792	0.770849918896
...			
25	6.000008483339	-0.000008483318	0.500004659653
26	6.000004241704	-0.000004241636	0.500004018472
27	6.000002120830	-0.000002120873	0.499994921199
28	6.000001060372	-0.000001060458	0.499979864670
29	6.000000530207	-0.000000530165	0.500019871456

Figures 13 & 14. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Steffensen's method with the following parameters: $\rho = 6$, $d = 2$, $x_0 = 5.2$ (left), $x_0 = 11$ (right), $\epsilon = 1 \times 10^{-6}$.

Figures 15 and 16 show the results of Steffensen's method on the function $(x - 6)^3$ with $x_0 = 2$ and $x_0 = 17$. Note that in these two tests, the convergence criteria was relaxed to $|x_k - \rho| < 1 \times 10^{-3}$. When more aggressive convergence was required, the method ran into a division by zero error. This occurs because $f(x_k)$ gets so close to 0 that a machine cannot compute the θ_k required by Steffensen's method. If the diminution of $f(x_k)$ was used as the convergence criterion, the methods would converge, but the computed root would not be all that close to the true root. This issue is only exacerbated as the multiplicity is pushed higher. Thus, for problems of this form, Steffensen's method converges linearly, but the convergence to the true root is incredibly slow.

Iteration	x	Delta x	Relative Error
1	2.013029315961	0.013029315961	0.996742671010
2	2.026171648998	0.013142333037	0.996703679541
3	2.039429257182	0.013257608184	0.996663769289
4	2.052804469344	0.013375212163	0.996622907901
5	2.066299688108	0.013495218764	0.996581061501
...			
107	5.995173322558	0.002413148990	0.666684137743
108	5.996782177553	0.001608854995	0.666674433018
109	5.997854773936	0.001072596383	0.666670115934
110	5.998569846027	0.000715072090	0.666668188279
111	5.999046563044	0.000476717017	0.666667347792

Iteration	x	Delta x	Relative Error
1	16.999267008729	-0.000732991271	0.999933364430
2	16.998533873312	-0.000733135417	0.999933346884
3	16.997800593682	-0.000733279630	0.999933329329
4	16.997067169773	-0.000733423909	0.999933311765
5	16.996333601519	-0.000733568254	0.999933294192
...			
3866	6.003521579540	-0.001760716081	0.666675966768
3867	6.002347734256	-0.001173845284	0.666670801813
3868	6.001565160469	-0.000782573786	0.666668497697
3869	6.001043441615	-0.000521718854	0.666667498761
3870	6.000695628055	-0.000347813561	0.666666964858

Figures 15 & 16. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Steffensen's method with the following parameters: $\rho = 6$, $d = 3$, $x_0 = 2$ (left), $x_0 = 17$ (right), $\epsilon = 1 \times 10^{-3}$.

5.1.4 Question 4

First, note that Regula Falsi will not work for even multiplicities. In these cases, $f(a_0)$ and $f(b_0)$ are both positive, so it cannot be concluded that there is a root in the interval. Consequently, only odd multiplicities were tested. As seen in Figures 17 and 18, interesting behavior occurs when the root is directly in the middle of the chosen initial interval. Any test where this is the case converges in one step. This is because the computed secant line

crosses directly through the true root. However, if the root was not directly in the middle of the initial interval, the method did not converge in any reasonable number of iterations (tests up to 100,000,000 iterations were tested). As in the prior question, if the convergence is checked with $f(x_k)$, the method converges, but the computed root is not very close to the true root.

Iteration	x	Delta x	Relative Error
1	2.400000000000	-0.600000000000	-0.000000000000

Iteration	x	Delta x	Relative Error
1	2.400000000000	-1.700000000000	0.000000000000

Figures 17 & 18. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Regula Falsi method with the following parameters: $\rho = 2.4$, $d = 3$, $a_0 = 1.8$, $b_0 = 3$, $\epsilon = 1 \times 10^{-6}$ (left); $\rho = 2.4$, $d = 9$, $a_0 = 0.7$, $b_0 = 4.1$, $\epsilon = 1 \times 10^{-6}$ (right).

Using Secant method with $d > 1$, all tests converge at what appears to be a linear or superlinear rate. Figure 19 shows the results of Secant method on the function $f(x) = (x - 2.4)^2$ with initial guesses $x_0 = 2$ and $x_1 = 2.1$. The relative error appears to converge to the golden ratio, which implies superlinear convergence. However, the rate at which the computed root approaches ρ seems to be linear. Figure 20 shows the results of Secant method on the function $f(x) = (x - 2.4)^7$ with initial guesses $x_0 = 4$ and $x_1 = 5$. Even when the progression of the initial guesses heads away from the the true root, the method still converges. In this case, the relative error approaches a different constant, about 0.89865. This constant gets closer to 1 as the chosen multiplicity gets higher, signifying weaker convergence.

Iteration	x	Delta x	Relative Error
1	2.228571428571	0.128571428571	0.571428571429
2	2.290909090909	0.062337662338	0.636363636364
3	2.333333333333	0.042424242424	0.611111111111
4	2.358620689655	0.025287356322	0.620689655172
5	2.37468085106	0.015847395451	0.617021276596
...			
23	2.39999579182	0.000002732216	0.618033988726
24	2.399997267784	0.000001688602	0.618033988713
25	2.399998311398	0.000001043613	0.618033988685
26	2.399998956387	0.000000644989	0.618033988658
27	2.399999355011	0.000000398625	0.618033988692

Iteration	x	Delta x	Relative Error
1	3.965422839206	-1.034577160794	0.602085707387
2	3.934872950040	-0.030549889165	0.980484576818
3	3.728354256237	-0.206518693803	0.865448997718
4	3.610333731516	-0.118020524721	0.911152823754
5	3.481778996627	-0.128554734890	0.893785712533
...			
132	2.400001383634	-0.00000156040	0.898653712489
133	2.400001243408	-0.00000140226	0.898653712564
134	2.400001117393	-0.00000126015	0.898653712684
135	2.400001004150	-0.00000113244	0.898653712500
136	2.400000902383	-0.00000101767	0.898653712752

Figures 19 & 20. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Secant method with the following parameters: $\rho = 2.4$, $d = 2$, $x_0 = 2$, $x_1 = 2.1$, $\epsilon = 1 \times 10^{-6}$ (left); $\rho = 2.4$, $d = 7$, $x_0 = 4$, $x_1 = 5$, $\epsilon = 1 \times 10^{-6}$ (right).

5.1.5 Question 5

When $d = 1$, Regula Falsi converges in one step in every case, no matter the ρ or the starting interval (as long as $a_0 \geq \rho \geq b_0$). As seen in Figures 21 and 22, the relative error is 0 and the method jumps straight to the true root ρ in one iteration.

Iteration	x	Delta x	Relative Error
1	5.000000000000	-7.000000000000	0.000000000000

Iteration	x	Delta x	Relative Error
1	10.000000000000	-5.000000000000	0.000000000000

Figures 21 & 22. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Regula Falsi method with the following parameters:

$\rho = 5$, $d = 1$, $a_0 = 1.5$, $b_0 = 12$, $\epsilon = 1 \times 10^{-6}$ (left); $\rho = 10$, $d = 1$, $a_0 = 2$, $b_0 = 15$, $\epsilon = 1 \times 10^{-6}$ (right).

When using Secant method with $d > 1$, all tests also converge in a single step. Figures 23 and 24 show this trend using Secant method with differing ρ values and initial guesses.

Iteration	x	Delta x	Relative Error	Iteration	x	Delta x	Relative Error
1	3.000000000000	1.500000000000	-0.000000000000	1	8.000000000000	-2.000000000000	0.000000000000

Figures 23 & 24. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Secant method with the following parameters: $\rho = 3$, $d = 1$, $x_0 = 1$, $x_1 = 1.5$, $\epsilon = 1 \times 10^{-6}$ (left); $\rho = 8$, $d = 1$, $x_0 = 6$, $x_1 = 10$, $\epsilon = 1 \times 10^{-6}$ (right).

Thus, when the function is of the form $f(x) = (x - \rho)$, both Regula Falsi and Secant method converge in a single iteration. This makes sense considering that this is just a linear function.

5.1.6 Question 6

When we use Modified Newton's method but we change m so that $m \neq d$, we observe interesting behavior. When $m < d$, the method converges linearly for all choices of x_0 . An example is given in Figure 25, where $d = 3$ and $m = 2$. The relative error being is constant at $\frac{1}{3}$. When $d < m < 2d$, the method also converges linearly for all initial guesses. An example of this is given in Figure 26, where $d = 3$ and $m = 5$. The relative error is steady around $-\frac{2}{3}$, which is of course less than 1 in magnitude.

Iteration	x	Delta x	Relative Error	Iteration	x	Delta x	Relative Error
1	9.66666666667	-11.33333333333	0.33333333333	1	-2.66666666667	-16.66666666667	-0.66666666667
2	5.88888888889	-3.77777777778	0.33333333333	2	8.44444444444	11.11111111111	-0.66666666667
3	4.62962962963	-1.25925925925	0.33333333333	3	1.03703703703	-7.40740740740	-0.66666666667
4	4.20987654321	-0.41975308642	0.33333333333	4	5.97530864197	4.93827160493	-0.66666666667
5	4.06995884773	-0.13991769547	0.33333333333	5	2.68312757201	-3.29218106995	-0.66666666667
...				...			
16	4.00000039492	-0.00000789839	0.33333333333	48	4.00000003528	0.00000008821	-0.66666667226
17	4.00000013164	-0.00000263280	0.33333333258	49	3.99999997647	-0.00000005881	-0.66666666247
18	4.00000004380	-0.00000087760	0.33333333582	50	4.00000001568	0.00000003920	-0.66666679252
19	4.00000001462	-0.00000029253	0.33333333333	51	3.99999998954	-0.00000002613	-0.66666676105
20	4.00000000487	-0.00000009751	0.33333333333	52	4.00000000697	0.00000001742	-0.66666669498

Figures 25 & 26. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Modified Newton's method with the following parameters: $\rho = 4$, $d = 3$, $m = 2$, $x_0 = 21$, $\epsilon = 1 \times 10^{-8}$ (left); $\rho = 4$, $d = 3$, $m = 5$, $x_0 = 14$, $\epsilon = 1 \times 10^{-8}$ (right).

However, when $m > 2d$, the method diverges, no matter what is chosen for x_0 . When running tests with parameters fitting this criteria, overflow errors were encountered every single time.

5.2 Three Distinct Roots

Consider the generic cubic polynomial used in the problem with three distinct roots 0, $\rho > 0$, and $-\rho$ with the following form and properties:

$$f(x) = x(x - \rho)(x + \rho) = x^3 - \rho^2 x, \quad \rho > 0$$

$$f'(x) = 3x^2 - \rho^2 = 3(x^2 - \alpha^2) = 3(x - \alpha)(x + \alpha), \quad \alpha = \rho/\sqrt{3} > 0$$

5.2.1 Question 1

Consider the problem where $\rho = 3$, $\alpha = 3/\sqrt{3}$, and $\xi_{\pm} = \pm\sqrt{\frac{9}{5}}$. We have three cases to consider for the initial guess: $x > \rho$, $\alpha < x < \rho$, and $-\xi < x < \xi$. Figures 27 and 28 show two examples of when $x > \rho$. In Figure 27, $x_0 = 5$ and in Figure 28, $x_0 = 57$. Thus x_0 's are tested that are close to and far from ρ . In both cases, however, the method converges quadratically and monotonically to ρ . Quadratic convergence can be seen in the dramatic push of x_k closer to ρ and in the convergence of the relative error to 0.

Iteration	x	Delta x	Relative Error
1	3.787878787879	-1.212121212121	0.393939393939
2	3.192836276669	-0.595042511210	0.244753735772
3	3.016171075355	-0.176665201314	0.083859093497
4	3.000129127940	-0.016041947415	0.007985117679
5	3.000000008336	-0.000129119604	0.000064557487

Iteration	x	Delta x	Relative Error
1	38.035120147874	-18.964879852126	0.648798521257
2	25.409439015612	-12.625681132262	0.639627862586
3	17.018704359666	-8.390734655946	0.625571409882
4	11.464550632908	-5.554153726758	0.603804061755
5	7.821559365935	-3.642991266974	0.569617877550
...			
7	4.060682837733	-1.422579409787	0.427132832544
8	3.309188635863	-0.751494201870	0.291499612197
9	3.038549666506	-0.270638969357	0.124680088575
10	3.000721414724	-0.037828251781	0.018713903122
11	3.000000260074	-0.000721154651	0.000360505084

Figures 27 & 28. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Standard Newton's method with the following parameters: $\rho = 3$, $\alpha = 3/\sqrt{3}$, $\xi_{\pm} = \pm\sqrt{\frac{9}{5}}$, $x_0 = 5$ (left), $x_0 = 57$ (right), $\epsilon = 1 \times 10^{-6}$.

Figures 29 and 30 show two examples of when $\alpha < x < \rho$. Here $\alpha = 3/\sqrt{3} \approx 1.732$. In Figure 29, $x_0 = 2.5$ and in Figure 30, $x_0 = 1.75$. Thus x_0 's are tested that are both towards the middle of this range and very close to α . In both cases again, the method converges quadratically to ρ . Quadratic convergence can be seen in the dramatic push of x_k closer to ρ and in the convergence of the relative error to 0. Notably, in every test case from this range, the first iteration pushes x_k above ρ and through the following iterations, x_k decreases monotonically to ρ . The closer x_0 is to α , the higher the computed root is pushed on the first iteration. This can be seen in the figures as $x_0 = 2.5$ is only pushed to about 3.2, while $x_0 = 1.75$ is pushed to 57.2.

Iteration	x	Delta x	Relative Error
1	3.205128205128	0.705128205128	-0.410256410256
2	3.018147906052	-0.186980299076	0.088471042006
3	3.000162381554	-0.017985524498	0.008947674380
4	3.000000013182	-0.000162368372	0.000081180525

Iteration	x	Delta x	Relative Error
1	57.166666666667	55.416666666667	-43.333333333334
2	38.146128679449	-19.020537987218	0.648851606390
3	25.483290729824	-12.662837949625	0.639708883299
4	17.067707532903	-8.415583196921	0.625696109255
5	11.496871372407	-5.570836160495	0.603998295567
...			
8	4.068306433798	-1.428175666336	0.427924732062
9	3.312642815700	-0.755663618098	0.292652750006
10	3.039330973938	-0.273311841761	0.125801623973
11	3.000750511155	-0.038580462783	0.019081936704
12	3.000000281469	-0.000750229686	0.000375036659

Figures 29 & 30. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Standard Newton's method with the following parameters: $\rho = 3$, $\alpha = 3/\sqrt{3}$, $\xi_{\pm} = \pm\sqrt{\frac{9}{5}}$, $x_0 = 2.5$ (left), $x_0 = 1.75$ (right), $\epsilon = 1 \times 10^{-6}$.

Figures 31 and 32 show two examples of when $-\xi < x < \xi$. Here, $\xi_{\pm} = \pm\sqrt{\frac{9}{5}} \approx 1.342$. In Figure 31, $x_0 = 1.3$ and in Figure 32, $x_0 = 0.3$. When the given x_0 is in this range, the computed root will converge to 0 rather than ρ . The convergence to 0 looks to be quadratic due to the rapid increase in accuracy of the computed root and the convergence of the relative error to 0.

Iteration	x	Delta x	Relative Error
1	-1.118066157761	-2.418066157761	-0.860050890585
2	0.532464981337	1.650531139097	-0.476237454859
3	-0.037048891164	-0.569513872500	-0.069579958236
4	0.000011306076	0.037060197239	-0.000305166367
5	-0.000000000000	-0.000011306076	-0.000000000028

Iteration	x	Delta x	Relative Error
1	-0.006185567010	-0.306185567010	-0.020618556701
2	0.000000052593	0.006185619604	-0.000008502606

Figures 31 & 32. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Standard Newton's method with the following parameters: $\rho = 3$, $\alpha = 3/\sqrt{3}$, $\xi_{\pm} = \pm\sqrt{9/5}$, $x_0 = 1.3$ (left), $x_0 = 0.3$ (right), $\epsilon = 1 \times 10^{-6}$.

5.2.2 Question 2

Consider the case when $x = \pm\xi$. That is, $x_0 = \sqrt{9/5} \approx 1.34164$. As ξ is a "cycling point" of Newton's method for this $f(x)$, the theory says that the method should cycle between ξ and $-\xi$. However, due to the limitations of computers, this behavior is not observable through the code. When setting $x_0 = \sqrt{9/5}$, the machine sets x_0 to a floating point number close to ξ , but it is not exactly equal. Consequently, the method cycles back and forth between numbers close to ξ in magnitude before ultimately converging quadratically to 0, as seen in Figure 33. This highlights that the floating point number used to represent ξ was actually less than ξ by some small amount. If instead it was represented by some $\xi + \epsilon$, the method would converge to $\rho = 3$.

Iteration	x	Delta x	Relative Error
1	-1.341640786500	-2.683281573000	-1.000000000000
2	1.341640786500	2.683281573000	-1.000000000000
3	-1.341640786500	-2.683281573000	-1.000000000000
4	1.341640786500	2.683281572999	-1.000000000000
5	-1.341640786499	-2.683281572998	-0.999999999999
...			
19	-1.240494698904	-2.564440679704	-0.936967759179
20	0.870947284887	2.111441983790	-0.702096740644
21	-0.196496642496	-1.067443927382	-0.225612555324
22	0.001707963849	0.198204606344	-0.008692076501
23	-0.000000001107	-0.001707964956	-0.000000648254

Figure 33. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Standard Newton's method with the following parameters: $\rho = 3$, $\alpha = 3/\sqrt{3}$, $\xi_{\pm} = \pm\sqrt{9/5}$, $x_0 = \sqrt{9/5}$, $\epsilon = 1 \times 10^{-6}$.

5.2.3 Question 3

As partially discussed in Question 1, if x_0 is such that $\alpha < x_0 < \rho$, but x_0 is just greater than α , say $\alpha + \epsilon$, Newton's method will move far away from x_0 yet still converge to ρ . We have that $\alpha = 3/\sqrt{3} \approx 1.73205$. Figure 34 shows the results of Standard Newton's method when $x_0 = 1.73206$. At the first iteration, the computed root jumps to above 100,000, before monotonically decreasing and converging to $\rho = 3$ in 31 iterations. The closer that x_0 is to α (while still remaining greater than α), the farther away x_1 will be from x_0 .

Iteration	x	Delta x	Relative Error
1	108786.593541192575	108784.861481192580	-85795.537281884448
2	72524.395712513011	-36262.197828679564	0.666657474273
3	48349.597169252273	-24174.798543260738	0.666652878013
4	32233.064820866908	-16116.532348385364	0.666645983543
5	21488.709942626025	-10744.354878240883	0.666635641661
...			
27	3.716504547977	-1.152924242784	0.383274587145
28	3.165121599556	-0.551382948421	0.230454363510
29	3.012082768232	-0.153038831324	0.073174970836
30	3.000072317094	-0.012010451138	0.005985142867
31	3.000000002615	-0.000072314479	0.000036156510

Figure 34. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Standard Newton's method with the following parameters: $\rho = 3$, $\alpha = 3/\sqrt{3}$, $\xi_{\pm} = \pm\sqrt{\frac{9}{5}}$, $x_0 = 1.73206$, $\epsilon = 1 \times 10^{-6}$.

5.2.4 Question 4

In a similar vein to previous questions, it is possible to make Newton's method fail by producing numbers that are undefined or too large for the floating point system. As discussed in Question 2, if x_0 is set equal to ξ or $-\xi$, the method will converge to 0 eventually, even though it is supposed to cycle between ξ and $-\xi$. This is due to the previously discussed limitations of the floating point system. Additionally, if $x_0 = \alpha + \epsilon$ where ϵ is very small, the method fails.

5.2.5 Question 5

When using Regula Falsi, convergence can be observed for a wide range of test cases. Two such intervals are shown in Figures 35 and 36. In both cases, the convergence is linear, as the relative error converges to a constant $C < 1$. However, if the bounds of the interval are far from ρ , the convergence becomes very slow. If the bounds become too far away, this causes problems for the method and we may not see convergence.

Iteration	x	Delta x	Relative Error
1	2.876712328767	-0.623287671233	-0.246575342466
2	2.973232499886	0.096520171119	0.217114167592
3	2.994364110092	0.021131610207	0.210549729467
4	2.998821201231	0.004457091138	0.209159296693
5	2.999753786537	0.000932585307	0.208868102781
6	2.999948588840	0.000194802303	0.208807265346
7	2.999989265629	0.000040676789	0.208794561340
8	2.999997758750	0.000008493121	0.208791908776
9	2.999999532046	0.000001773296	0.208791354849

Iteration	x	Delta x	Relative Error
1	1.019417475728	-18.980582524272	-0.116504854369
2	1.039094546748	0.019677071020	0.990065008260
3	1.059028399183	0.019933852435	0.989834363303
4	1.079215839506	0.020187440323	0.989599311852
5	1.099653283501	0.020437443995	0.989359843539
...			
401	2.999998861420	0.000000046367	0.960869585830
402	2.999998905973	0.000000044553	0.960869585131
403	2.999998948783	0.000000042810	0.960869584329
404	2.999998989917	0.000000041135	0.960869583475
405	2.999999029442	0.000000039525	0.960869582540

Figures 35 & 36. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Regula Falsi method with the following parameters: $\rho = 3$, $\alpha = 3/\sqrt{3}$, $\xi_{\pm} = \pm\sqrt{\frac{9}{5}}$, $a_0 = 2.5$, $b_0 = 3.5$, $\epsilon = 1 \times 10^{-6}$ (left), $\rho = 3$, $\alpha = 3/\sqrt{3}$, $\xi_{\pm} = \pm\sqrt{\frac{9}{5}}$, $a_0 = 1$, $b_0 = 20$, $\epsilon = 1 \times 10^{-6}$ (right).

On the other hand, Secant method seems to work for some pairs of initial guesses and not work for other pairs. Figures 37 and 38 show the results of Secant method for two pairs of initial guesses for which the method converges. The convergence rate appears to be superlinear - it is faster than linear, but doesn't quite seem quadratic. In running a wide range of tests, it seems that those that converge tend to have either $x_0 > \alpha$ or $x_1 > \alpha$, but there were a few rogue exceptions to this.

Iteration	x	Delta x	Relative Error
1	9.963779527559	8.063779527559	-6.330708661417
2	1.990774454550	-7.973005073009	-0.144924970909
3	2.078673671666	0.087899217116	0.912904288331
4	4.920777542797	2.842103871131	-2.084796107227
5	2.405460285669	-2.515317257128	-0.309530750482
...			
7	3.169876755239	0.528993990655	-0.473039827905
8	2.967212537450	-0.202664217789	-0.193007351146
9	2.997357380187	0.030144842737	0.080598485127
10	3.000043927173	0.002686546985	-0.016622585030
11	2.999999941900	-0.000043985273	-0.001322646453

Iteration	x	Delta x	Relative Error
1	4.200000000000	-0.800000000000	0.600000000000
2	3.535871156662	-0.664128843338	0.446559297218
3	3.191805185802	-0.344065970860	0.357931535253
4	3.040032262348	-0.151772923454	0.208713138700
5	3.003520939858	-0.036511322489	0.087952557555
6	3.000069300748	-0.003451639111	0.019682457057
7	3.000000121832	-0.000069178916	0.001758015126

Figures 37 & 38. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Secant method with the following parameters: $\rho = 3$, $\alpha = 3/\sqrt{3}$, $\xi_{\pm} = \pm\sqrt{\frac{9}{5}}$, $x_0 = 1.8$, $x_1 = 1.9$, $\epsilon = 1 \times 10^{-6}$ (left), $\rho = 3$, $\alpha = 3/\sqrt{3}$, $\xi_{\pm} = \pm\sqrt{\frac{9}{5}}$, $x_0 = 7$, $x_1 = 5$, $\epsilon = 1 \times 10^{-6}$ (right).

Steffensen's method exhibits some interesting behavior with problems of this format. When x_0 is chosen such that $x_0 > \rho$, as it is in Figure 39, the method converges monotonically to ρ . When $\alpha < x_0 < \rho$, as in Figure 40, the method also converges to ρ , but with different behavior than Newton's. While Newton's jumped up and then decreased monotonically to ρ , Steffensen's increases in a few steps to just above ρ and then settles down to ρ . The convergence for both of these tests appears to be faster than linear due to the rapid addition of zeros to the computed root by the end. Thus, the convergence is likely quadratic, but could also potentially be superlinear.

Iteration	x	Delta x	Relative Error	Iteration	x	Delta x	Relative Error
1	4.989564309940	-0.010435690060	0.994782154970	1	1.943848410179	0.193848410179	0.844921271857
2	4.979050143434	-0.010514166506	0.994715342222	2	2.163711386621	0.219862976442	0.791826307358
3	4.968456081044	-0.010594062390	0.994646895418	3	2.458482132697	0.294770746076	0.647525099158
4	4.957780662291	-0.010675418753	0.994576755430	4	3.329409513306	0.870927380608	-0.608307746052
5	4.947022384015	-0.010758278276	0.994504860282	5	3.280447558343	-0.048961954962	0.851364477999
...				...			
93	3.034468045699	-0.041030225554	0.456540860166	10	3.034566265677	-0.041064305632	0.457040917173
94	3.008985276494	-0.025482769205	0.260684245707	11	3.009031226769	-0.025535038908	0.261272850639
95	3.000719539954	-0.008265736540	0.080079890100	12	3.000726687215	-0.008304539555	0.080463843209
96	3.00004892784	-0.000714647170	0.006799878601	13	3.000004990209	-0.000721697006	0.006867065571
97	3.00000000227	-0.00004892557	0.000046479756	14	3.000000000237	-0.00004989972	0.000047405296

Figures 39 & 40. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Steffensen's method with the following parameters: $\rho = 3$, $\alpha = 3/\sqrt{3}$, $\xi_{\pm} = \pm\sqrt{\frac{9}{5}}$, $x_0 = 5$ (left), $x_0 = 1.75$ (right), $\epsilon = 1 \times 10^{-6}$.

Figures 41 and 42 further demonstrate some difference in behavior between Steffensen's method and Newton's method for problems of this type. In Figure 41, $x_0 = \sqrt{\frac{9}{5}}$, the cycling point from Newton's method. Here, we see no cycling whatsoever - just quick quadratic-like convergence to $\rho = 3$. In Figure 42, $x_0 = 0.6$. Whereas in Newton's method, this initial guess would've converged to 0, with Steffensen's method, it converges to $\rho = 3$, again at a seemingly quadratic rate.

Iteration	x	Delta x	Relative Error	Iteration	x	Delta x	Relative Error
1	1.531674892236	0.190034105737	0.885408357737	1	1.138728600503	0.538728600503	0.775529749790
2	1.716716284366	0.185041392129	0.873977914598	2	1.348229302349	0.209500701846	0.887442152766
3	1.908079331260	0.191363046894	0.850880172044	3	1.537896676890	0.189667374542	0.885173302317
4	2.121317644806	0.213238313546	0.804712631924	4	1.722965289573	0.185068612683	0.873423027116
5	2.395035196879	0.273717552073	0.688490897245	5	1.914759609729	0.191794320155	0.849812758738
6	2.989590843752	0.594555646873	0.017206217939	...			
7	3.001113514715	0.011522670963	-0.106974541315	8	3.036433212648	0.629995731299	-0.061380581663
8	3.000011684107	-0.001101830608	0.010492997721	9	3.009921775864	-0.026511436784	0.272327778495
9	3.000000001297	-0.000011682811	0.000110989546	10	3.000871697348	-0.009050078516	0.087856988519
				11	3.000007172944	-0.000864524405	0.008228708792
				12	3.000000000489	-0.000007172455	0.000068139409

Figures 41 & 42. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Steffensen's method with the following parameters: $\rho = 3$, $\alpha = 3/\sqrt{3}$, $\xi_{\pm} = \pm\sqrt{\frac{9}{5}}$, $x_0 = \sqrt{\frac{9}{5}}$ (left), $x_0 = 0.6$ (right), $\epsilon = 1 \times 10^{-6}$.

5.3 Three Distinct Roots: Scaling

Consider testing the methods on the function $\tilde{f}(x) = \sigma f(x)$, where $\sigma > 0$.

5.3.1 Question 1

Figures 43 and 44 show the results of Newton's method with $\sigma = 5$ and $\sigma = 100$, respectively. Both use $x_0 = 8$, testing the case where $x_0 > \rho$. Interestingly, the tables of data are exactly the same.

Iteration	x	Delta x	Relative Error
1	5.595628415301	-2.404371584699	0.519125683060
2	4.125715264948	-1.469913150352	0.433696617864
3	3.338959884773	-0.786755380175	0.301106234700
4	3.045485349822	-0.293474534951	0.134190952574
5	3.000999124631	-0.044486225192	0.021965855697
6	3.000000498737	-0.000998625893	0.000499174412

Figures 43 & 44. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Steffensen's method with the following parameters: $\rho = 3$, $\alpha = 3/\sqrt{3}$, $\xi_{\pm} = \pm\sqrt{\frac{9}{5}}$, $x_0 = 8$, $\sigma = 5$ (left), $\sigma = 100$ (right), $\epsilon = 1 \times 10^{-6}$.

Figures 45 and 46 show the results of Newton's method with $\sigma = 4$ and $\sigma = 100$, respectively. Both use $x_0 = 1.75$, testing the case where $\alpha < x_0 < \rho$. Again, the tables of data are exactly the same.

Iteration	x	Delta x	Relative Error
1	57.166666666667	55.416666666667	-43.333333333333
2	38.146128679449	-19.020537987218	0.648851606390
3	25.483290729824	-12.662837949625	0.639708883299
4	17.067707532903	-8.415583196921	0.625696109255
5	11.496871372407	-5.570836160495	0.603998295567
...			
8	4.068306433798	-1.428175666336	0.427924732062
9	3.312642815700	-0.755663618098	0.292652750006
10	3.039330973938	-0.273311841761	0.125801623973
11	3.000750511155	-0.038580462783	0.019081936704
12	3.000000281469	-0.000750229686	0.000375036659

Figures 45 & 46. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Steffensen's method with the following parameters: $\rho = 3$, $\alpha = 3/\sqrt{3}$, $\xi_{\pm} = \pm\sqrt{\frac{9}{5}}$, $x_0 = 1.75$, $\sigma = 4$ (left), $\sigma = 100$ (right), $\epsilon = 1 \times 10^{-6}$.

Figures 47 and 48 show the results of Newton's method with $\sigma = 4$ and $\sigma = 50$, respectively. Both use $x_0 = 0.5$, testing the case where $-\xi < x_0 < \xi$. Once again, the tables of data are exactly the same.

Iteration	x	Delta x	Relative Error
1	-0.030303030303	-0.530303030303	-0.060606060606
2	0.000006185554	0.030309215857	-0.000204123290
3	-0.000000000000	-0.000006185554	-0.000000000000

Figures 47 & 48. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Steffensen's method with the following parameters: $\rho = 3$, $\alpha = 3/\sqrt{3}$, $\xi_{\pm} = \pm\sqrt{\frac{9}{5}}$, $x_0 = 0.5$, $\sigma = 4$ (left), $\sigma = 50$ (right), $\epsilon = 1 \times 10^{-6}$.

In all three cases, all of the convergence data remains the same even after scaling. Evidently, scaling $f(x)$ by σ has no impact on convergence.

5.3.2 Question 2

Similar to Newton's method, the behavior of both Regula Falsi and Secant method is unchanged by scaling the function $f(x)$. However, the behavior for Steffensen's method is impacted by the scaling. Figures 49 and 50 show an example of this when $x_0 = 6$, i.e. $x_0 > \rho$. When $\sigma = 5$, Steffensen's takes 4,771 iterations to converge, but when $\sigma = 50$, Steffensen's takes 457,665 iterations to converge.

Iteration	x	Delta x	Relative Error	Iteration	x	Delta x	Relative Error
1	5.999758489756	-0.000241510244	0.999919496585	1	5.999997536343	-0.000002463657	0.999999178781
2	5.999516944435	-0.000241545321	0.999919478411	2	5.999995072682	-0.000002463661	0.999999178779
3	5.999275364024	-0.000241580410	0.999919460228	3	5.999992609017	-0.000002463665	0.999999178777
4	5.999033748512	-0.000241615512	0.999919442038	4	5.999990145349	-0.000002463668	0.999999178775
5	5.998792097886	-0.000241650626	0.999919423839	5	5.999987681677	-0.000002463672	0.999999178773
...				...			
4767	3.008102798365	-0.008907953638	0.476333930651	457661	3.001415260851	-0.001031075638	0.578522561248
4768	3.002312263472	-0.005790534893	0.285366038769	457662	3.000596278430	-0.000818982421	0.421320514627
4769	3.00224810690	-0.002087452782	0.097225377962	457663	3.000132133352	-0.000464145078	0.221596732989
4770	3.00002281455	-0.00022529235	0.010148338312	457664	3.00007516209	-0.000124617143	0.056883513061
4771	3.00000000237	-0.00002281218	0.000103797854	457665	3.00000025383	-0.000007490826	0.003377159096

Figures 49 & 50. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Steffensen's method with the following parameters: $\rho = 3$, $\alpha = 3/\sqrt{3}$, $\xi_{\pm} = \pm\sqrt{\frac{9}{5}}$, $x_0 = 6$, $\sigma = 5$ (left), $\sigma = 50$ (right), $\epsilon = 1 \times 10^{-6}$.

Figures 51 and 52 show an example of this when $x_0 = 2$, i.e. $\alpha < x_0 < \rho$. When $\sigma = 5$, Steffensen's takes 133 iterations to converge, but when $\sigma = 50$, Steffensen's takes 15,269 iterations to converge.

Iteration	x	Delta x	Relative Error	Iteration	x	Delta x	Relative Error
1	2.004539264639	0.004539264639	0.995460735361	1	2.000040485338	0.000040485338	0.999959514662
2	2.009086905220	0.004547640581	0.995431622343	2	2.000080971184	0.000040485846	0.999959512515
3	2.013643096462	0.004556191241	0.995402027418	3	2.000121457538	0.000040486354	0.999959510368
4	2.018208016210	0.004564919749	0.995371939171	4	2.000161944399	0.000040486861	0.999959508221
5	2.022781845542	0.004573829332	0.995341345817	5	2.000202431768	0.000040487369	0.999959506073
...				...			
129	3.007721539736	-0.008786400603	0.467747010050	15265	3.001258367071	-0.001005475464	0.555854504721
130	3.002122771683	-0.005598768053	0.274915593958	15266	3.000490336446	-0.000768030626	0.389660900154
131	3.000190665992	-0.001932105691	0.089819359001	15267	3.000092256790	-0.000398079656	0.188149973963
132	3.000001643026	-0.000189022966	0.008617300199	15268	3.000003714014	-0.000088542775	0.040257351725
133	3.000000000123	-0.000001642903	0.000074753379	15269	3.000000006206	-0.000003707808	0.001670989229

Figures 51 & 52. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Steffensen's method with the following parameters: $\rho = 3$, $\alpha = 3/\sqrt{3}$, $\xi_{\pm} = \pm\sqrt{\frac{9}{5}}$, $x_0 = 2$, $\sigma = 5$ (left), $\sigma = 50$ (right), $\epsilon = 1 \times 10^{-6}$.

When using Steffensen's method, the greater the scaling factor, the greater the number of iterations until convergence.

5.4 Three Distinct Roots: Root Coalescing

Consider the function $\hat{f}(x) = x(x - \rho_1)(x - \rho_2)$, where $\rho_1 > 0$, $\rho_2 > 0$, and $0 < \rho_1 < \rho_2$.

5.4.1 Question 1

As $\rho_1 \rightarrow 0$, the root at 0 is closer and closer to becoming a double root. That is, the function is transforming into $\hat{f}(x) = x^2(x - \rho_2)$. The convergence rate at 0 degrades from quadratic to linear. Figures 53 and 54 demonstrate this trend. In Figure 53, $\rho_1 = 3$ and we can see that the convergence to 0 is quadratic. However, in Figure 54, $\rho_1 = 0.00001$ and we can see that convergence to 0 is now linear.

Iteration	x	Delta x	Relative Error
1	-0.004930847865	0.095069152135	0.049308478653
2	-0.000012915067	0.004917932799	0.002619238489
3	-0.000000000089	0.000012914978	0.000006887963

Iteration	x	Delta x	Relative Error
1	-0.050482985325	0.049517014675	0.504829853250
2	-0.025364543231	0.025118442094	0.502437466170
3	-0.012711709751	0.012652833480	0.501160601831
4	-0.006361410869	0.006350298882	0.500437076837
5	-0.003180230094	0.003181180776	0.499925277445
...			
11	-0.000044967604	0.000049715822	0.474925825978
12	-0.000020234087	0.000024733518	0.449970304061
13	-0.000008112434	0.000012121652	0.400929112447
14	-0.000002509517	0.000005602917	0.309342050260
15	-0.000000419314	0.000002090203	0.167089460064

Figures 53 & 54. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Newton's method with the following parameters: $\rho_1 = 3$ (left), $\rho_1 = 0.00001$ (right) $\rho_2 = 5$, $x_0 = -0.1$, $\epsilon = 1 \times 10^{-6}$.

5.4.2 Question 2

As $\rho_1 \rightarrow 0$ and $\rho_2 \rightarrow 0$, the root at 0 is closer and closer to becoming a triple root. That is, the function is transforming into $\hat{f}(x) = x^3$. The convergence rate at 0 degrades to an even slower linear rate. Figure 55 demonstrates this trend. In this example, ρ_1 is kept at $\rho_1 = 0.00001$, but we now have $\rho_2 = 0.0000001$. It can be observed that the convergence rate now is an even slower linear, with the relative error converging to around 0.5.

Iteration	x	Delta x	Relative Error
1	-0.066665544518	0.033334455482	0.666655445178
2	-0.044442574233	0.022222970285	0.666649834700
3	-0.029627260765	0.014815313468	0.666641419317
4	-0.019750385202	0.009876875563	0.666628796992
5	-0.013165801617	0.006584583585	0.666609865192
...			
23	-0.000006353402	0.000004222570	0.600739334614
24	-0.000003676043	0.000002677359	0.578594408184
25	-0.000002036644	0.000001639399	0.554031651661
26	-0.000001077050	0.000000959594	0.528835567767
27	-0.000000541810	0.000000535240	0.503049880474

Figure 55. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Newton's method with the following parameters: $\rho_1 = 0.00001$, $\rho_2 = 0.0000001$, $x_0 = -0.1$, $\epsilon = 1 \times 10^{-6}$.

5.4.3 Question 3

The Secant method also suffers as the roots coalesce to 0. Figure 56 shows the results of Secant method with $\rho_1 = 3$ and $\rho_2 = 5$, while Figure 57 shows the results of Secant method with $\rho_1 = 0.00001$ and $\rho_2 = 0.0000001$. Both tests use initial guesses of $x_0 = -1.5$ and $x_1 = -0.5$. Superlinear convergence can be observed in Figure 56, but when the roots coalesce to 0 in Figure 57, we now see linear convergence, highlighted by the relative error converging to around 0.63.

Iteration	x	Delta x	Relative Error
1	-0.218978102190	0.281021897810	0.437956204380
2	-0.045116530578	0.173861571611	0.206032156308
3	-0.004754392325	0.040362138254	0.105380273345
4	-0.000112115056	0.004642277268	0.023581364090
5	-0.00000283724	0.000111831333	0.002530646269

Iteration	x	Delta x	Relative Error
1	-0.461537923672	0.038462076328	0.923075847344
2	-0.319828108106	0.141709815566	0.692961708458
3	-0.249155652196	0.070672455910	0.779029878492
4	-0.185778759593	0.063376892603	0.745633333844
5	-0.140901467375	0.044877292218	0.758436904646
...			
40	-0.000005043047	0.000002243554	0.692098716929
41	-0.000003422777	0.000001620270	0.678712046537
42	-0.000002274219	0.000001148557	0.664437003807
43	-0.000001477758	0.000000796462	0.649786775643
44	-0.000000938349	0.000000539408	0.634981805645

Figures 56 & 57. Iteration count, computed root approximation, the change in the root estimate, and the relative error using Secant method with the following parameters: $\rho_1 = 3$, $\rho_2 = 5$ (left), $\rho_1 = 0.00001$, $\rho_2 = 0.0000001$ (right), $x_0 = -1.5$, $x_1 = -0.5$, $\epsilon = 1 \times 10^{-6}$.

6 Conclusions

This report examined the performance and behavior of several root-finding methods, including Regula Falsi, Secant, Newton's (standard and modified), and Steffensen's methods. Through numerical experiments involving polynomial functions, we explored convergence rates, numerical stability, and sensitivity to root multiplicities, initial guesses, and scaling factors.

The results confirmed theoretical expectations, such as linear convergence of standard Newton's method for multiple roots and quadratic convergence when using the modified version. Steffensen's method demonstrated robustness but faced challenges with scaling. Regula Falsi showed guaranteed convergence under specific conditions, albeit slowly in certain cases. The Secant method exhibited superlinear convergence but lacked guaranteed stability.

Overall, these experiments highlighted the strengths and limitations of each method, emphasizing the importance of selecting appropriate algorithms based on the problem's characteristics and computational constraints.

7 References

In the creation of this report, I consulted with Dan Mazus and Brayton Link regarding algorithm development.